

ATIVIDADE REMOTA — MÁQUINAS DE TURING

Teoria da Computação

Estudo, simulação e reflexão sobre os limites computacionais

Etapa 1 — Introdução

Após assistir ao vídeo e estudar o material disponibilizado, responda às questões abaixo.

1. O que é uma Máquina de Turing?

É um modelo matemático e teórico abstrato de computação idealizado por Alan Turing em 1936. O modelo descreve um dispositivo capaz de manipular símbolos em uma fita contínua de acordo com uma tabela de regras pré-definidas. Não é uma máquina física, mas uma ferramenta formal usada na teoria da computação e da complexidade para definir matematicamente o que pode ou não ser processado por um computador.

2. Quais são os principais componentes de uma Máquina de Turing?

Fita, Cabeçote, Registrador de Estado e Tabelas de regras.

3. Qual é a importância das Máquinas de Turing para a computação?

Elas criaram a base da ciência da computação. O conceito da máquina universal inspirou o computador moderno capaz de armazenar programas na memória e ajudou a provar matematicamente quais problemas são impossíveis de resolver.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Ela é a representação matemática exata de um algoritmo. Segundo a tese de Church e Turing, qualquer problema que possa ser resolvido por um algoritmo de programação também pode ser resolvido por uma Máquina de Turing.

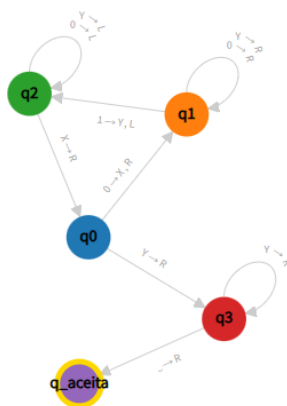
Etapa 2 — Simulação

Descrição do Funcionamento:

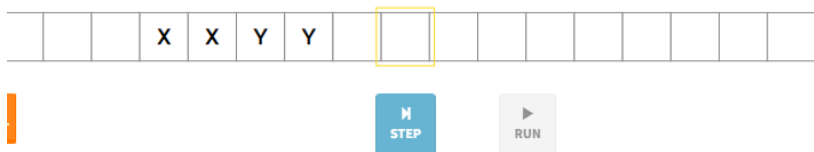
A máquina opera por marcação de pares. Ela substitui o primeiro 0 por X, avança até o primeiro 1 e o substitui por Y. Em seguida, retorna e repete o ciclo. A palavra é aceita se todos os números formarem pares exatos. Se sobram números ou a ordem estiver errada, a máquina trava e rejeita a entrada.

Exemplo de Palavra Aceita (0011):

A máquina marca o primeiro par de zero e um (Fita: X0Y1). Em seguida, repete o processo para o segundo par (Fita: XXY1). Como não sobrou nenhum número original na fita, a máquina faz a leitura final e aceita a palavra.

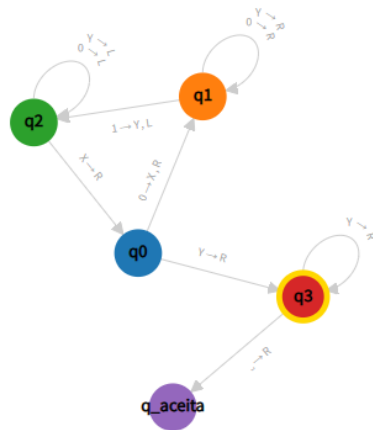


```
1 input: '0011'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     1: {R: q3}
8
9   q1:
10    0: {R: q1}
11    1: {R: q1}
12    1: {write: Y, L: q2}
13
14   q2:
15    0: {L: q2}
16    1: {L: q2}
17    X: {R: q0}
18
19   q3:
20    Y: {R: q3}
21    ' ': {R: q_aceita}
22
23 q_aceita:
```



Exemplo de Palavra Rejeitada (011):

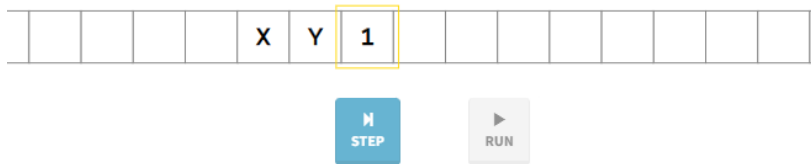
A máquina marca o único zero e o primeiro um correspondente (Fita: XY1). Ao tentar continuar, ela esbarra no número 1 que sobrou na fita. Como a regra exige pares exatos, a máquina trava e rejeita a palavra imediatamente.



```

1 input: '011'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     Y: {R: q3}
8
9   q1:
10    0: {R: q1}
11    Y: {R: q1}
12    1: {write: Y, L: q2}
13
14  q2:
15    0: {L: q2}
16    Y: {L: q2}
17    X: {R: q0}
18
19  q3:
20    Y: {R: q3}
21    ' ': {R: q_aceita}
22
23 q_aceita:

```



Etapa 3 — Registro da Simulação

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	01	ACEITA	ACEITA	q0 → q1 → q2 → q0 → q3 → q_aceita
2	000111	ACEITA	ACEITA	q0 → q1 → q2 (repete o ciclo 3 vezes) → q0 → q3 → q_aceita
3	0111	REJEITA	REJEITA (Travou)	q0 → q1 → q2 → q0 → q3 → (Erro ao ler o número 1)

Teste 1:

```
graph TD
    q0((q0)) -- "0" --> q1((q1))
    q1 -- "1" --> q0
    q0 -- "X" --> q2((q2))
    q2 -- "Y" --> q0
    q0 -- "Y" --> q3((q3))
    q3 -- "Y" --> q_aceita(((q_aceita)))
    q3 -- "X" --> q_aceita
    q1 -- "0" --> q1
    q2 -- "1" --> q2
    q3 -- "Y" --> q3
```

```
1 input: '01'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     Y: {R: q3}
8
9   q1:
10    0: {R: q1}
11    Y: {R: q1}
12    1: {write: Y, L: q2}
13
14  q2:
15    0: {L: q2}
16    Y: {L: q2}
17    X: {R: q0}
18
19  q3:
20    Y: {R: q3}
21    ' ': {R: q_aceita}
22
23  q_aceita:
```

				X	Y									
--	--	--	--	---	---	--	--	--	--	--	--	--	--	--

▶

▶

Teste 2:

```
graph TD
    q0((q0)) -- "0" --> q1((q1))
    q1 -- "1" --> q0
    q0 -- "X" --> q2((q2))
    q2 -- "Y" --> q0
    q0 -- "Y" --> q3((q3))
    q3 -- "Y" --> q_aceita(((q_aceita)))
    q3 -- "X" --> q_aceita
    q1 -- "0" --> q1
    q2 -- "1" --> q2
    q3 -- "Y" --> q3
```

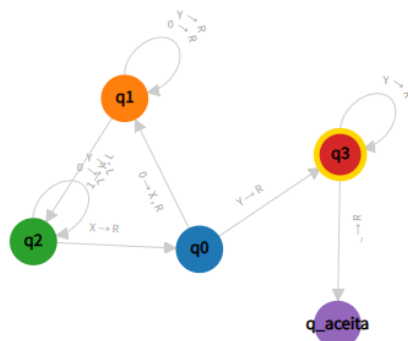
```
1 input: '000111'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     Y: {R: q3}
8
9   q1:
10    0: {R: q1}
11    Y: {R: q1}
12    1: {write: Y, L: q2}
13
14  q2:
15    0: {L: q2}
16    Y: {L: q2}
17    X: {R: q0}
18
19  q3:
20    Y: {R: q3}
21    ' ': {R: q_aceita}
22
23  q_aceita:
```

	X	X	X	Y	Y	Y								
--	---	---	---	---	---	---	--	--	--	--	--	--	--	--

STEP

RUN

Teste 3:



```
1 input: '0111'
2 blank: ' '
3 start state: q0
4 table:
5   q0:
6     0: {write: X, R: q1}
7     Y: {R: q3}
8
9   q1:
10    0: {R: q1}
11    Y: {R: q1}
12    1: {write: Y, L: q2}
13
14   q2:
15    0: {L: q2}
16    Y: {L: q2}
17    X: {R: q0}
18
19   q3:
20    Y: {R: q3}
21    ' ': {R: q_aceita}
22
23 q_aceita:
```



Descrição da Máquina de Turing criada:

A máquina utiliza a lógica de marcação para validar se a palavra segue a regra 0^n1^n (quantidades exatas e em ordem). Ela funciona em ciclos: lê e substitui o primeiro 0 por X, avança para a direita ignorando outros zeros até achar o primeiro 1, que é substituído por Y. Depois, retorna à esquerda até o último X e repete o processo. Quando não há mais zeros, a máquina entra em um estado de varredura final (q3) para verificar se sobraram apenas marcações Y. Se a fita terminar apenas em espaços em branco, a palavra é aceita. Se sobrar qualquer número sem par, a máquina trava e rejeita a entrada.

Etapa 4 - Reflexão sobre os limites computacionais

Não. A própria lógica tem limites matemáticos. Para um computador resolver algo, ele precisa de instruções exatas, mas existem problemas que simplesmente não podem ser divididos em um passo a passo ou que caem em contradições. Essa limitação não é culpa da máquina ou de falta de potência, mas da própria natureza da questão. Nem tudo pode ser transformado em código.